Learning to Break Deep Perceptual Hashing: The Use Case NeuralHash

Lukas Struppek*
lukas.struppek@cs.tu-darmstadt.de
Department of Computer Science, Technical University of Darmstadt
Darmstadt, Germany

Dominik Hintersdorf*
dominik.hintersdorf@cs.tu-darmstadt.de
Department of Computer Science, Technical University of Darmstadt
Darmstadt, Germany

Daniel Neider
neider@mpi-sws.org
Max Planck Institute for Software Systems; and Safety and Explainability of Learning Systems, Carl von Ossietzky University of Oldenburg
Kaiserslautern, Germany

Kristian Kersting
kersting@cs.tu-darmstadt.de
Department of Computer Science and Centre for Cognitive Science, Technical University of Darmstadt; and Hessian Center for AI
Darmstadt, Germany

ABSTRACT

Apple recently revealed its deep perceptual hashing system NeuralHash to detect child sexual abuse material (CSAM) on user devices before files are uploaded to its iCloud service. Public criticism quickly arose regarding the protection of user privacy and the system's reliability. In this paper, we present the first comprehensive empirical analysis of deep perceptual hashing based on NeuralHash. Specifically, we show that current deep perceptual hashing may not be robust. An adversary can manipulate the hash values by applying slight changes in images, either induced by gradient-based approaches or simply by performing standard image transformations, forcing or preventing hash collisions. Such attacks permit malicious actors easily to exploit the detection system: from hiding abusive material to framing innocent users, everything is possible. Moreover, using the hash values, inferences can still be made about the data stored on user devices. In our view, based on our results, deep perceptual hashing in its current form is generally not ready for robust client-side scanning and should not be used from a privacy perspective.[1]

CCS CONCEPTS

• **Computing methodologies** → **Machine learning**; • **Security and privacy** → *Software and application security*.

KEYWORDS

perceptual hashing, client-side scanning, neuralhash, neural networks, deep learning, privacy

*Both authors contributed equally to this research.
[1]An extended version of this paper, including an appendix with experiment details and additional results, is available at https://arxiv.org/abs/2111.06628.

Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.
FAccT '22, June 21–24, 2022, Seoul, Republic of Korea
© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9352-2/22/06…$15.00
https://doi.org/10.1145/3531146.3533073

ACM Reference Format:
Lukas Struppek, Dominik Hintersdorf, Daniel Neider, and Kristian Kersting. 2022. Learning to Break Deep Perceptual Hashing: The Use Case NeuralHash. In *2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT '22), June 21–24, 2022, Seoul, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3531146.3533073

1 INTRODUCTION

In 2020, the US National Center for Missing & Exploited Children (NCMEC) received over 21 million reports of online child sexual exploitation, an increase of 28% from 2019 [19]. Child sexual abuse material (CSAM) is no niche phenomenon and must be pursued and prevented. With millions of users uploading pictures and other media to online platforms, there is an unmanageable flood of data. One approach to efficiently analyze this amount of data is to transform the images into a lower-dimensional space by extracting image-specific features. These compact representations enable an efficient search in and comparison of high-dimensional data. In this paper, we will focus on deep perceptual hashing that computes short bit sequences of fixed-length, the so-called hashes or fingerprints, for multimedia content. Hashing generally describes a deterministic transformation of data into short bit sequences. Perceptual hashing aims to assign similar hashes to images with similar visual features. Recent deep perceptual hashing algorithms use neural networks for feature extraction. Perceptual hashing algorithms differ significantly from traditional cryptographic hashing algorithms, which aim to produce widely differing hashes for minor input changes, the so-called avalanche effect.

Apple recently announced its NeuralHash [26] system, a deep perceptual hashing algorithm for client-side content scanning. The approach focuses on identifying CSAM content in user files uploaded to Apple's iCloud service. Apple has made several assurances about privacy and security, such as a low risk of accounts being falsely flagged and restricting its access to private data. More detailed information on NeuralHash is provided in the official technical summary [26]. According to Apple, only images uploaded to the iCloud servers will be hashed and compared against a hash database of known CSAM material. The hash databases are provided by NCMEC and other child protection agencies and are only

available in encrypted form on the user devices. While Apple commissioned various expert opinions on the security of the system from independent researchers [5, 20, 41], there has been a lot of public criticism of the NeuralHash approach [1, 34, 43]. Criticism is directed not only at possible privacy violations but also at the system's reliability. So far, however, there has been a lack of comprehensive analyses, at least publicly, of the core of the perceptual hashing process, the hash computation itself.

The European Union (EU) presented in 2020 their strategy [9] to become more effective in fighting child sexual abuse. The strategy specifically refers to end-to-end encryption and calls for technical solutions that allow companies to detect and report CSAM material transferred in encrypted communication systems. The subsequent EU regulation 2021/1232 [39], which also faces criticism [6], establishes temporarily limited rules with the sole purpose of allowing service providers to use specific technologies for the processing of personal and other data to the extent necessary to detect and report CSAM material. Without the regulation stating specific technical details, it can be assumed that hash-based client-side scanning approaches could be a way to achieve this. Similarly, the UK governments' Online Safety Bill [18] suggests a comparable direction and establishes new ways to regulate online content to prevent the distribution of harmful material.

A recent paper [1] by cybersecurity researchers and encryption system inventors outlines and critiques the potential security and privacy risks of client-side scanning technologies. The authors argue that even if the initial goal of these systems is only the detection of clearly illegal content, tremendous pressure to expand the scope of application will arise with time. People would then have little chance to resist the expansion of the system or prevent its abuse.

In this paper, we investigate the perceptual hashing components of NeuralHash, in particular, the embedding neural network and the hashing step. It might be to some extent common knowledge that neural networks are susceptible to various kinds of attacks. However, we are convinced that it is important to demonstrate that this susceptibility is not only interesting from a researcher's point of view but actually affects systems used by millions of users who might not be aware of the risks of these systems. We further want to emphasize that by using neural networks with the ability to compute gradients, most of the attacks are rather easy to perform, exposing various risks to manipulate the systems. Our focus lies on NeuralHash because it is the first prominent representative to shift content detection from the server-side to user devices. This approach poses additional major risks and insecurities, such as causeless algorithmic surveillance of users. We show that deep perceptual hashing has various downsides when applied to real-world large-scale image detection. Our research aims to point out drawbacks of deep perceptual image hashing, support the development of more robust systems and encourage a discussion on the general deployment of this technology. NeuralHash merely acts as a current real-world example in this case.

We proceed as follows. In Section 2, we first introduce deep perceptual hashing together with the specifics of NeuralHash and a general overview of attacks against neural networks. Our first adversarial setting in Section 3 then sheds light on the possibility to create hash collisions with minor visual image changes. In Sections 4 and 5, we show that the hash computation is susceptible to gradient-based and gradient-free image transformations. In our fourth adversarial setting in Section 6, we demonstrate that a hash value itself actually contains information about its corresponding image and might leak information about the content on user devices. We conclude our work by a discussion of our results and their implications in Section 7, followed by a summary in Section 8.

Before diving into the details, we want to make the following two statements:

(1) We explicitly condemn the creation, possession, and distribution of child pornography and abusive material and strongly support the prosecution of related crimes. With this work, we in no way intend to provide instructions on how to bypass or manipulate CSAM filters. In turn, we want to initiate a well-founded discussion about the effectiveness and the general application of client-side scanning based on deep perceptual hashing.
(2) We have no intention to harm Apple Inc. itself or their intention to stop the distribution of CSAM material. NeuralHash merely forms the empirical basis of our work to critically examine perceptual hashing methods and the risks they may induce in real-world scenarios. Even though this system is not in production yet, we contacted Apple and made sure they are aware of the possible issues with NeuralHash.

2 BACKGROUND

We start by formally defining a perceptual hashing system, describing the characteristics of NeuralHash, and giving an overview of security-related attacks against neural networks.

2.1 Perceptual Hashing and NeuralHash

Perceptual hashing algorithms, e.g., Apple's NeuralHash [26], Microsoft's PhotoDNA [38] and Facebook's PDQ [17], aim to compute similar hashes for images with similar contents and more divergent hashes for different contents. In recent years, various deep hashing algorithms based on convolutional neural networks have been proposed [35, 36, 51, 55]. They all rely on deep neural networks to first extract unique features from an image and then compute a hash value based on these features. In the following, we introduce the basic deep perceptual hashing approach, before describing the specifics of NeuralHash. Figure 1 gives an overview of the hash computation steps, which we now explain in detail.

We define a (perceptual) hash function $H\colon \mathbb{R}^{H\times W\times C} \to \{0,1\}^k$ that maps an image x to a k-bit binary hash. Let $h_i(x)$ further denote the partial hash function for the i-th hash bit, i.e., $H(x) = \big(h_1(x), \ldots, h_k(x)\big)$. Perceptual hashing algorithms usually consist of two components. First, a shared feature extractor $M(x)\colon \mathbb{R}^{H\times W\times C} \to \mathbb{R}^m$ extracts visual features from an image x and encodes them in a feature vector $z \in \mathbb{R}^m$. This resulting feature vector z is an abstract numeric interpretation of the image's characteristic features.

Next, locality-sensitive hashing (LSH) [21, 27] is used to assign close feature vectors to buckets with similar hash values. Among other LSH methods, random projection can be used to quickly convert the extracted features into a bit representation. For each of the k bits, a (random) hyperplane is defined in the hashing matrix $B \in \mathbb{R}^{m\times k}$.

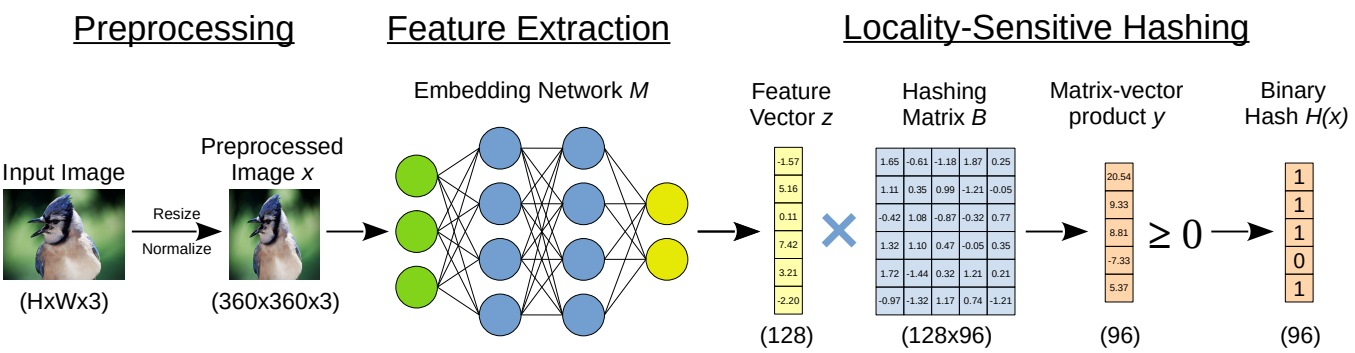


Figure 1: NeuralHash pipeline as deployed on user devices. The pipeline consists of an embedding network and a locality-sensitive hashing (LSH) step. The embedding network maps the preprocessed images into an abstract feature representation vector. LSH then maps each vector into a specific bucket by checking its position relative to the hyperplanes defined in the hashing matrix.

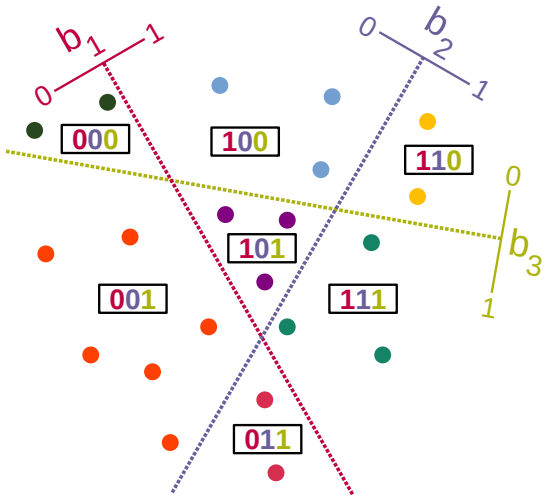


Figure 2: Locality-sensitive hashing (LSH) scheme. Each hyperplane divides the space into two parts, with a bit state {0, 1} assigned to each side. The polytopes constrained by the hyperplanes are called buckets. Each bucket is assigned a unique binary hash code based on its relative position to each hyperplane. All data points in the same bucket are assigned the same hash code.

The bit value $h_i(x)$ is set by checking on which side of the i-th hyperplane feature vector z lies. In practice, the feature vector z is first transformed into $y = B \cdot z$ with $y \in \mathbb{R}^k$ by calculating the product of the hashing matrix and the embedding. The real-valued vector y is then finally converted to a bit vector by applying a Heaviside step function to each element y_i. The result is a binary hash vector $H(x)$ containing k bits. For the figures in this paper, we represent the computed hashes in their equivalent, compact hexadecimal representation. We visualized the basic concept of LSH in a 2D case with three random hyperplanes in Figure 2. Points with the same color are assigned to the same bucket and result in the same hash.

To compare the similarity and distance between two hashes, we define the normalized Hamming distance of two given binary hashes x and y as $\delta(H(x), H(y)) = \frac{1}{k} \sum_{i=1}^{k} |h_i(x) - h_i(y)|$. In other words, δ describes what percentage of bits in the two hashes differ. It allows quantifying the distance between two hashes and analyzing their proximity. For brevity, we refer to it simply as the Hamming distance in this paper.

Figure 1 illustrates the algorithmic pipeline of NeuralHash, which we now describe in more detail. NeuralHash [26] expects resized RGB input images with shape $360 \times 360 \times 3$ within the pixel range $[-1, 1]$. The embedding network M is based on a modified MobileNetV3 [25] architecture with 1.8 million parameters. The network has been trained in a self-supervised training scheme with a contrastive loss. The idea behind contrastive learning [23] is to create a positive and negative pair of samples for each training image, the so-called anchors. For the positive pair, the anchor is a transformed version of the training image itself so that it remains perceptually similar, e.g., by scaling or cropping. The negative pair is built up by selecting a perceptually different image, e.g., from another class. A neural network is then trained to output close feature vectors z for the positive pairs and more distant vectors for negative pairs.

Closeness is usually described by the cosine of the angles between the feature vectors. Unfortunately, Apple does not state any additional details on training data, loss functions, or hyperparameters. The embedding network produces feature vectors z of length $m = 128$. The hashing matrix B has shape 128×96 and, consequently, defines 96 hyperplanes. Therefore, the final hashes have a fixed length of $k = 96$ bits. For our experiments, we extracted the neural network as well as the hashing matrix used by NeuralHash from a Mac running macOS Big Sur Version 11.6.

2.2 Attacks Against Neural Networks

To put our work into context, we briefly introduce common attacks against neural networks. Most similar to our gradient-based hash attacks in Sections 3 and 4 are adversarial examples [8, 10, 14, 44, 45]. They apply precisely tuned perturbations to images that are hardly perceivable to humans but lead to misclassifications. Most attacks

compute the gradients of the output with respect to a model's input and perform one [22] or many [37], possibly greedy [40], optimization steps to force the model's outputs to specific values. While our attacks are not directly based on a specific algorithm for adversarial examples, our first two adversarial settings are motivated by their general findings and algorithmic approaches and can be seen as a form of creating targeted and untargeted adversarial examples in a perceptual hashing setting. In both settings, the adversary perturbs an image only until the optimization goal is achieved, a similar approach was introduced by Carlini and Wagner [8]. Our Edges-Only and Few-Pixels evasion attacks are further in the spirit of sparse adversarial examples [13, 40, 56] that try to change as few pixels as possible.

While much work has been done on the general vulnerability of neural networks to adversarial attacks, little work has been published on the vulnerability of deep hashing functions. As a first step, targeted [4, 46, 47, 52] and untargeted [53] adversarial attacks against deep hashing-based retrieval systems were proposed. The goal of these attacks is to manipulate input images in a way that the target system retrieves objects with a specific target label or objects that are semantically irrelevant to the original inputs. Another work [12] demonstrated the susceptibility of various image hashing functions against gradient-based collision attacks. Similar works investigated the robustness of non-deep perceptual hashing algorithms against adversarial attacks [24, 28] and their robustness [15] against visible image modifications. Other research has focused on recovering the original images from the supposedly anonymous real-valued image hashes by training a deep neural network to reconstruct inputs, given their hash values [48]. However, they only investigated the de-hashing of non-deep perceptual hashing algorithms.

Until now, there is no comprehensive work on the technical vulnerabilities from a machine learning perspective of NeuralHash or client-side scanning based on deep perceptual hashing yet. However, a few proof-of-concept implementations to create single hash collisions on NeuralHash exist [3, 30, 31] but did not provide any systematic or in-depth vulnerability analysis. Our work aims to fill this research gap and help to weigh the privacy benefits and potential threats. We improve the optimization procedure and further introduce an SSIM penalty term to reduce the visual conspicuities of our attacks. Overall, the attacks proposed in this paper focus primarily on demonstrating the actual vulnerabilities of deep perceptual hashing. However, each attack also acts as a baseline for future attacks and defenses.

3 ADVERSARY 1 – HASH COLLISION ATTACKS

In our first adversarial setting, we investigate the creation of hash collisions. We perturb images so that their computed hashes match predefined target hashes. This issue holds several explosive scenarios in reality. Given a set of target hashes from the (CSAM) hash database, an adversary can create fake images whose hashes match those in the database without containing any sensitive material at all. Distributed across many devices, this may lead to a large number of false-positive alarms in the (possibly partly human) detection system and, as a consequence, result in the framing of innocent users or distributed denial of service (DDoS) attacks.

Another worrisome scenario could be that service providers, governments, or other powerful organizations either add additional hashes to the database or manipulate ordinary images that show political or social content that is undesirable to them, such as government critics or LGBTQ+ supporters. By spreading such manipulated images through social media, they might end up on the devices of supporters of these groups. Identification, surveillance, and persecution of these people are then possible by detecting hash collisions with the database. This poses a great danger, especially for people in states with restricted fundamental rights or totalitarian regimes.

Even without access to the hash database, which governments, after all, might have, an adversary could simply collect its own CSAM material and compute its hashes. Assuming a large enough database, the adversary could obtain, at least, some share of the true database hashes. Figure 3 shows a manipulated image from a protest march that results in the same hash value as a non-related target image. For this paper, we created a surrogate hash database with dog images that acts as a list of images that should be detected by the system.

Technical Realization. In this setting, the adversary has full access to the perceptual hashing function with the ability to compute gradients and knows at least a share of the hashes from the database. The goal is to manipulate a given image so that its hash matches any hash from the hash dataset. To create hash collisions, we first computed the hash of an input image x_{orig} and took the target hash $\hat{H}$ from the (surrogate) hash database with the smallest Hamming distance to the computed hash of x_{orig}. After defining the target hash $\hat{H} = (\hat{h}_1, \dots, \hat{h}_k)$, we directly optimized the input image $x = x_{orig}$ such that its binary hash value approaches $\hat{H}$. For this task, we defined a Hinge loss

$$\mathcal{L}_{Hinge}(x, \hat{H}) = \frac{1}{k} \sum_{i=1}^{k} max\{0, d - y_i \cdot \psi(\hat{h}_i)\} \tag{1}$$

with $d \geq 0$ describing the margin to the hyperplanes and $y = B \cdot M(x)$ with $y \in \mathbb{R}^{96}$ being the real-valued hash output before binarization. The operation $\psi(\hat{h}_i) = sign(\hat{h}_i - 0.5)$ replaces each 0-bit in the hash vector by -1. In our experiments, we set $d = 0$ to optimize only until the hash value of the optimized image matches $\hat{H}$. This is achieved when the signs of y_i and $\psi(\hat{h}_i)$ match at all k positions. By setting $d > 0$, the distance to the LSH hyperplanes could be increased, leading to more robust image hashes. Alternatively, an MSE loss might be used for the optimization. However, in our experiments, we found the Hinge loss to facilitate the attacks.

Furthermore, we added a structural similarity (SSIM) [49] penalty term to reduce visual conspicuities between an optimized image and its original counterpart. The structural similarity (SSIM) is defined as

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}. \tag{2}$$

For two images x and y, the parameters μ_i and σ_i^2 denote the mean and variance of each image's pixels. Further, σ_{xy} denotes the covariance of x and y. The constants C_1 and C_2 are added for numerical stability and are set to $C_1 = 10^{-4}$ and $C_2 = 9 \cdot 10^{-4}$, respectively, in our experiments. The closer $SSIM \in [0, 1]$ is to 1, the

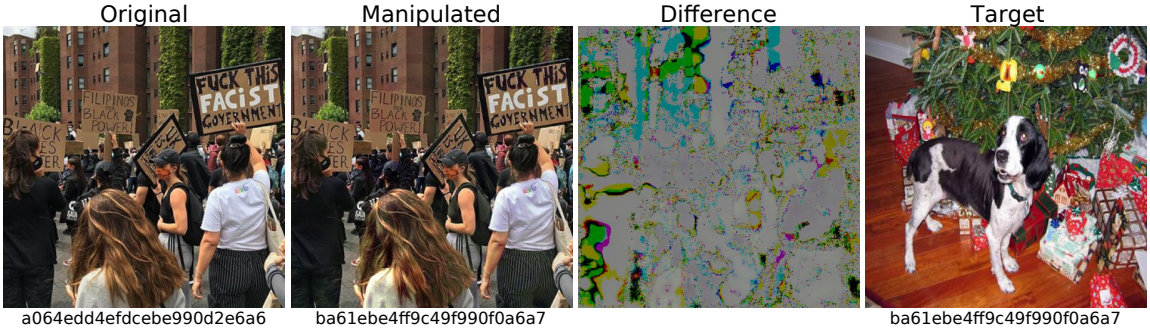


Figure 3: We manipulated the original image [50] to have the same hash as the target image (Adversary 1). The manipulated image is visually hardly distinguishable from the original since the (normalized) differences are small. Still, the manipulated image is assigned the same hash as the visually completely different target image. This demonstrates the practicability and danger of collision attacks.

SR	ℓ_2	ℓ_∞	SSIM	Steps
90.81%	20.8136 ± 7.97	0.3120 ± 0.22	0.9647 ± 0.03	1190 ± 1435

Table 1: Evaluation metrics (mean + std) for our hash collision attack computed on an ImageNet subset.

more similar x is to the original image without perturbations. We computed the SSIM, weighted by parameter λ, in each optimization step between the optimized image x and its unmodified counterpart x_{orig} to improve the image quality. We note that deviations from the original image can also be penalized with Lp-norm distances or more advanced similarity metrics, such as LPIPS [54].

In total, we attempted to solve the following optimization problem:

$$\begin{aligned} \min_{x} \quad & \mathcal{L}_{Hinge}(x, \hat{H}) - \lambda \cdot SSIM(x, x_{orig}) \\ \text{s.t.} \quad & H(x) = \hat{H} \\ & x \in [-1, 1]^{H \times W \times C}. \end{aligned} \tag{3}$$

Experimental Setup. For our analyses, we created a surrogate hash database by hashing all 20,580 dog images from the Stanford Dogs dataset [29]. We then performed our attacks by modifying the samples on the first 10,000 samples from the ImageNet ILSVRC2012 [11, 42] test split.

We used the Adam optimizer [32] to directly optimize x. We further set $\lambda = 100$ and stopped the optimization when either $H(x) = \hat{H}$ was satisfied or aborted after 10,000 iterations. If we did not stop the optimization process when $H(x) = \hat{H}$ is fulfilled, the image quality might be further improved due to the SSIM term.

Results. Table 1 states our collision attack results. The success rate (SR) indicates the share of images whose hashes have successfully been changed. We further state the mean ℓ_2 and ℓ_∞ distances between x_{orig} and its optimized counterpart x to quantify pixel-wise image changes. We also computed the mean SSIM values to take the image quality into account. Steps denote the mean number of optimization steps performed until a hash collision occurred.

We could force hash collisions in about 90% of all images, demonstrating the real applicability of the attack. The visual salience of the modifications varies in strength. Most manipulated images contain some patches of color, which are sometimes conspicuous but often hardly noticeable. Figure 3 shows such an example where the visual differences between the original and manipulated images are small. We expect that hyperparameter tuning and a careful selection of input images might further improve the success rate and visual indistinguishability in real scenarios.

In summary, we demonstrated that hash collisions can easily be forced in NeuralHash and might build the base for serious attacks targeting the service provider or, even worse, persecution of political opponents. While some of the introduced image changes are visible, they are barely noticeable in most images, as also the SSIM values indicate.

4 ADVERSARY 2 – GRADIENT-BASED EVASION ATTACKS

Next, we investigate how robust NeuralHash is and if we can change the hash of any image by perturbating it. This is also called a detection evasion attack and aims to evade detection of sensitive material through hash comparison by an automated system. A perceptual hashing algorithm used to identify sensitive material should be robust to small image changes to provide reliable results.
Two kinds of image manipulations are plausible: optimization-based approaches that compute image-specific perturbations and transformation-based approaches that transform an image using simple procedures provided by standard image editors. Non-robust perceptual hashing algorithms would make it easy to hide sensitive material from detection and call into question the overall effectiveness of such systems.

In this section, we follow an optimization-based approach and compute gradient-based perturbations that aim to alter the hash of an image with minimal differences from the original one. We propose three different attacks: Standard, Edges-Only, and Few-Pixels. Our Standard attack allows changes to all parts of an image. The Edges-Only attack first detects the edges in an image and then restricts the manipulation to the edge pixels. For the Few-Pixels attack, we attempt to change as few pixels of an image as possible.

Technical Realization. In our second setting, the adversary again has full access to the perceptual hashing function and tries to manipulate an input image so that its hash value differs from the original image's hash. The goal is to avoid detection by comparison with the hash database. To change the original hash $\tilde{H} = H(x_{orig}) =$

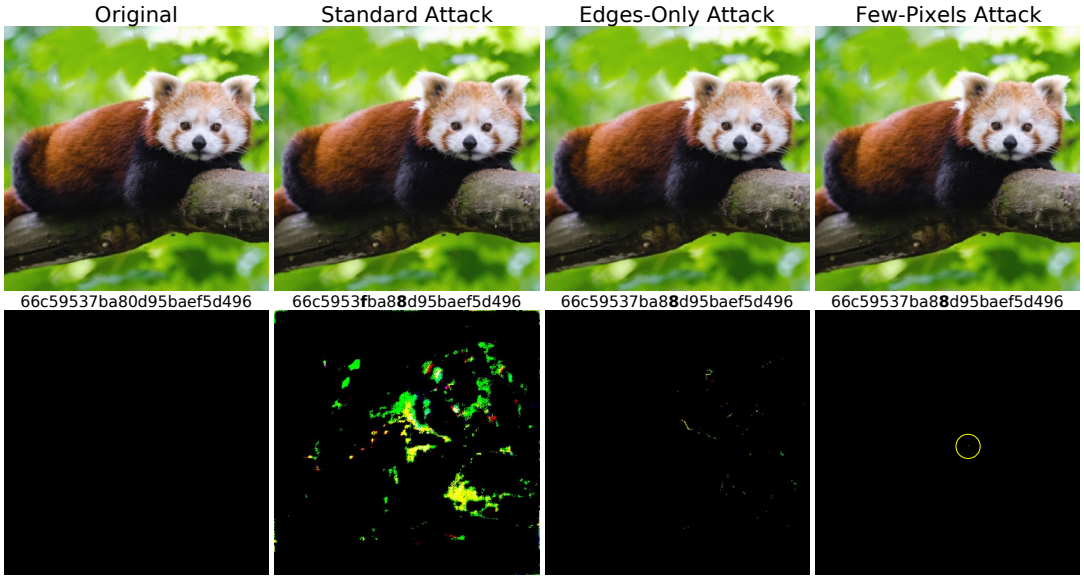


Figure 4: Visualization [2] of our gradient-based evasion attacks (Adversary 2). The added perturbations are hardly visible, even changing a single pixel leads to a hash change. Normalized differences are visualized below. Black marks unmodified image parts.

$(\tilde{h}_1, \dots \tilde{h}_k)$ of a given input image x_{orig}, we used a negative mean squared error (MSE) loss to increase the hash discrepancy between an image x_{orig} and its manipulated counterpart x as

$$\mathcal{L}_{MSE}(x, \tilde{H}) = -\frac{1}{k} \sum_{i=1}^{k} \left(\sigma(c\, \widetilde{y_i}) - \tilde{h}_i\right)^2 . \tag{4}$$

We make the binarization step differentiable by replacing it with a sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$. To push the sigmoid values closer to zero and one, we scaled its inputs by factor c. In our experiments, we set $c = 5$. For a stable optimization, we normalized the matrix-vector product $y = B \cdot M(x)$ by $\widetilde{y} = \frac{y}{max(\|y\|_2, \epsilon)}$ and set $\epsilon = 10^{-12}$ for numerical stability. For each attack, we optimized until a minimal Hamming distance δ_0 between x and x_{orig} is exceeded. As with Adversary 1, we also added an SSIM penalty term weighted by hyperparameter λ to reduce visual conspicuities.

The full optimization goal can be formulated as

$$\begin{aligned} \min_{x} \quad & \mathcal{L}_{MSE}(x, \tilde{H}) - \lambda * SSIM(x, x_{orig}) \\ \text{s.t.} \quad & \delta(H(x), \tilde{H}) > \delta_0 \\ & x \in [-1, 1]^{H \times W \times C}. \end{aligned} \tag{5}$$

For the Edges-Only attack, we first applied a Canny edge detector [7] in the greyscale version of an image. We then only allowed changes in the set of edge pixels by applying a binary mask to the gradients during optimization.

As for our Few-Pixels attack, we started by selecting a single pixel to optimize by taking the pixel with the highest absolute $\mathcal{L}_{MSE}$ loss gradient value across all three color channels. We then again applied a binary mask to the gradients and optimized for N steps. If the hash did not change, we added another pixel to the pixel set and again optimized for N steps on all pixels of the set. Additional pixels were greedily selected by their absolute $\mathcal{L}_{MSE}$ loss gradient values. We repeatedly added more pixels and modified the set of selected pixels until at least one hash bit flipped or a threshold of 150 pixels has been reached. A similar greedy approach is used by JSMA [40] to create adversarial examples.

Attack	Standard	Edges-Only	Few-Pixels
SR	100.00%	99.95%	98.21%
ℓ_2	0.7188 ± 0.28	1.3882 ± 1.37	2.9100 ± 2.06
ℓ_∞	0.0044 ± 0.00	0.0841 ± 0.07	0.8298 ± 0.25
SSIM	0.9999 ± 0.00	0.9996 ± 0.00	0.9989 ± 0.00
Steps	5.4006 ± 4.98	150.2414 ± 113.96	3095.0 ± 3901

Table 2: Evaluation metrics (mean + std) for our gradient-based evasion attacks computed on an ImageNet subset.

Experimental Setup. We performed our evasion attacks on the first 10,000 ImageNet test samples. We used the Adam optimizer to directly optimize an image x. We applied a decaying weight for the SSIM term and set $\lambda = 5 \cdot 0.99^{step}$, where $step$ denotes the number of optimization steps already performed. By decaying the weight, we avoided that the optimizer getting stuck in local minima in some cases. For the Standard and Edges-Only attacks, we performed a maximum of 1,000 optimization steps.

In the Few-Pixels attack, we set $\lambda = 0$ and, therefore, removed the SSIM term. The optimization is aborted after 150 pixels without a hash change. In all three attacks, we set the minimum Hamming distance $\delta_0 = 0$ and, consequently, stopped each attack when $H(x) \neq \tilde{H}$.

We also can force our attacks to produce larger perturbations and move a sample even further away from its original hash in terms of the Hamming distance by setting $\delta_0 > 0$. We repeated our Standard attack with minimal Hamming distances $\delta_0 \in [0, 0.5]$ and increased δ_0 in steps of 0.05.

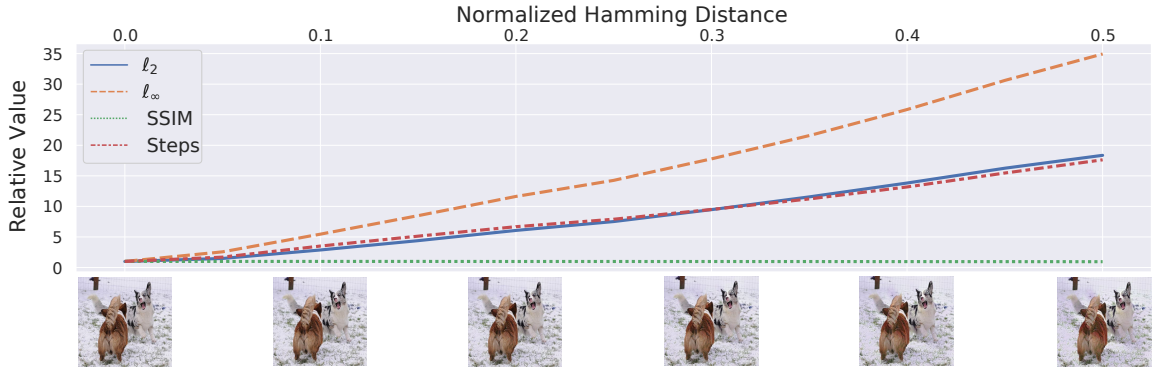


Figure 5: We performed the Standard evasion attack (Adversary 2) with a varying minimum Hamming distance δ_0, which can also be interpreted as the minimum percentage of hash bits to flip. Metrics are relative to the results for $\delta_0 = 0$, as stated in Table 2. For example, an ℓ_2 value of 10 means a ten times higher average ℓ_2 distance between x_{orig} and the manipulated image compared to the average distance when the first bit flips. Below the graph, we state the optimization results for a random image at increasing Hamming distances.

Results. Table 2 states the results for our gradient-based hash change attacks with $\delta_0 = 0$. We rely on the same metrics as used for our Adversary 1. The success rate (SR) indicates the share of images whose hashes could be changed before the maximum number of iterations or pixels has been reached. All three attacks were able to change the hash value of an image in the large majority of the cases. While the Standard Attack changed the hashes of all images, the Edges-Only and Few-Pixels attacks only failed in a few cases. Probably the optimization space in these attacks was too restricted, and the restrictions could be relaxed to achieve higher success rates.

The distances between the original and manipulated images are quite low, and the mean SSIM values are close to one, demonstrating that minor perturbations are sufficient to alter an image's hash. Figure 4 illustrates the effectiveness of our attacks and shows that the changes are hardly visually perceivable. The Standard attack only needs about five optimization steps on average to force at least a single hash bit to flip. In the Few-Pixels attack, only 21.5 pixels were changed on average, 0.017% of all pixels. All gradient-based attacks in this work can also be applied to images before resizing them into a square since most resizing operations, such as bilinear interpolation, are differentiable.

We plot the Standard attack results for increasing δ_0 in Figure 5. While the number of optimization steps required and the ℓ_2 and ℓ_∞ distances increase proportionally, the SSIM only decrease slightly, 0.96 on average for $\delta_0 = 0.5$, a flip of more than half of the bits, after all, indicating that even for larger Hamming distances the visual quality decreases only slightly. For all values of δ_0, we achieved a success rate of 100%.

Our results in this adversarial setting demonstrate that hash changes can easily be forced by gradient-based attacks. For humans, the induced changes are hardly perceivable. It is also possible to limit image manipulations to edges and even single pixels to keep the number of changes small. Our analysis shows that NeuralHash is in no way robust against gradient-based evasion attacks and questions the general robustness of neural network-based hashing approaches.

5 ADVERSARY 3 – GRADIENT-FREE EVASION ATTACKS

We now extend our analysis from the previous section and investigate the robustness of NeuralHash against basic, gradient-free transformations as provided by standard image editors. The more robust the hashing algorithm is, the less susceptible it is to image transformations, such as rotations, translations, or color changes. Many of these operations, e.g., flipping and color changes, can be reverted. This makes them particularly interesting to investigate since these transformations could be used to evade detection systems by first applying a transformation to bypass the system and then reverting them to reconstruct the original image. Unlike gradient-based attacks, the adversary does not need to have direct access to the hash algorithm or any expertise in computer science. Using a simple image editor is sufficient to perform the attacks.

Technical Realization. Unlike previous adversarial settings, we now investigate the case in which the adversary has black-box access to the perceptual hashing function and can only observe the input-output relation without the ability to compute gradients. The goal is still to avoid detection by comparison with the hash database. We applied different image transformations $T(x) : \mathbb{R}^{H\times W\times C} \rightarrow \mathbb{R}^{H\times W\times C}$ independently to the input images x. We then obtained the perceptual hash of each transformed image $T(x)$ and calculated the Hamming distance $\delta\left(H(T(x)), H(x)\right)$ between the hashes of the transformed and the original images. For transformations with additional hyperparameters, such as degrees in rotation, we computed results for varying parameter values to take the transformations' strength into account. All investigated transformations kept the image size and removed image parts were filled with black color.

Experimental Setup. We evaluated the robustness of NeuralHash on the 1,281,167 images from the ImageNet training split. To save computing resources, we varied most hyperparameters with an exponentially increasing step size. We investigated the following

transformations independently: translation, rotation, center cropping, downsizing, flipping, changes in the HSV color space, contrast changes, and JPEG compression.

Results. We plot the effects of various image transformations on the Hamming distance in Figures 6 and 7. Our experimental results show that NeuralHash is susceptible to most, but not all, image transformations. This non-robust behavior leads to significant hash changes, even if the content of an image was only slightly modified from a human perspective. We now discuss the average effects of different transformations in more detail.

As stated in Figure 7, image translations had a high impact on the hash calculation. Shifting an image by 32 pixels in horizontal and vertical directions resulted in a mean Hamming distance of roughly 11%. Larger image translations even further raised the Hamming distance. Image rotations also significantly increased the Hamming distances with rotations of only one degree already changing more than 5% of the hash bits, and rotating it by 32 degrees flipped even more than 30% of the bits. Similar non-robust behavior could be observed for center cropping and downsizing, where larger cropping windows and smaller image sizes increased the Hamming distance almost linearly. Furthermore, flipping the images, one of the simplest transformations without any loss of information and easy to revert, changed 29% of the bits for horizontally flipping and even 37% for vertical flipping.

While manipulating the hue of the images flipped 10% to 15% percent of the bits, varying the saturation had only minor influence and led to a hamming distance between 1% and 2%. Doubling the contrast and the brightness flipped roughly 37% and 22% of the hash bits. However, decreasing the values had almost no effect.

Investigating the effects of compression is especially interesting since most online services and messaging apps compress images during transmission. As can be seen in Figure 6f, compression had only a slight impact on the hashes when choosing moderate compression values. Compressing the images with a compression value of 92, which still resulted in reasonable compression and image quality, changed only roughly 2% of the bits.

In summary, our analysis demonstrated that NeuralHash is not robust against many basic image transformations. While the system showed stronger robustness against HSV space modifications and image compression, it is more susceptible to other transformations, such as rotations or flipping.

The most significant hash changes occurred when image information was lost or added, e.g., by cutting parts out or adding black borders. We assume that the embedding network has been trained in a contrastive learning setting, mainly based on color transformations and, therefore, shows larger robustness against these manipulations. However, an adversary could evade detection with small effort and little to low quality loss in the images. In reality, the reliability of such a system is therefore questionable.

6 ADVERSARY 4 – HASH INFORMATION EXTRACTION

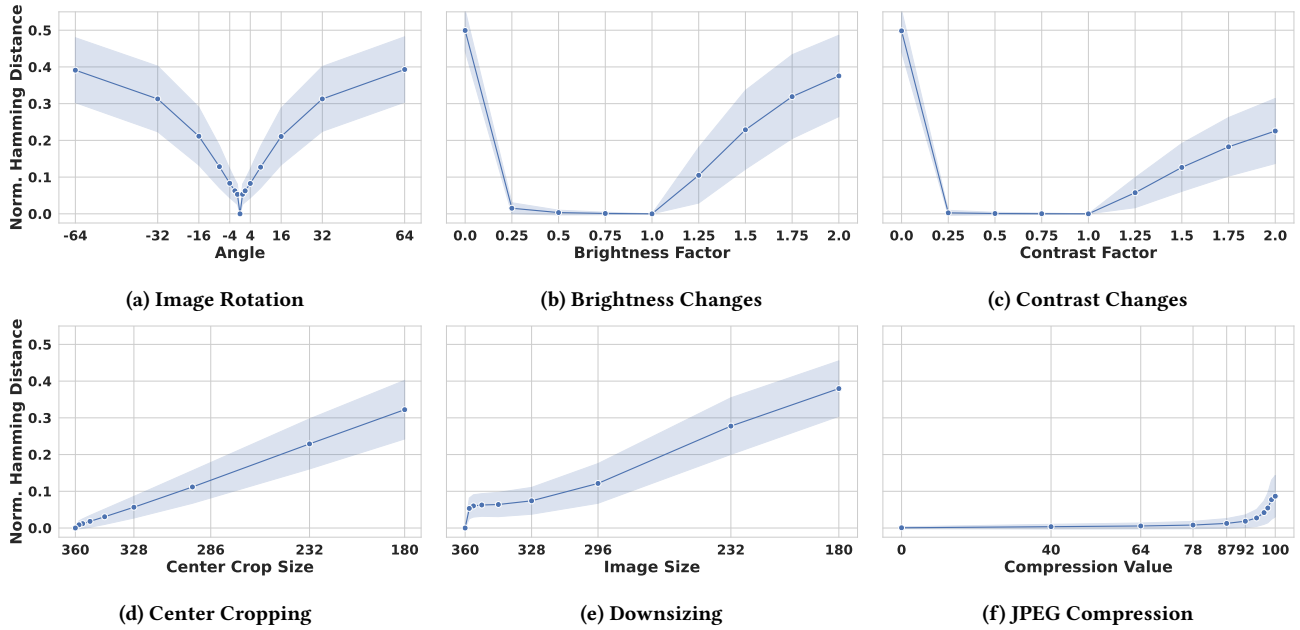
In our last adversarial setting, we want to investigate whether a hash value leaks information about its corresponding image. Since the embedding network is trained with contrastive learning, we expect that the extracted feature vector contains specific information about the image. Considering these features are to some degree included in the hash value, we assume that the hash value leaks at least approximate information on an image's content. Should this assumption turn out to be true, then NeuralHash and presumably also other perceptual hash-based systems would not completely anonymize user data and still leak details about the contents on user devices. This might not be critical for a single hash value due to prediction uncertainty, but in the case of larger hash sets from the same user, an adversary might expose the general content of the images.

Technical Realization. To verify our assumption, we took a classification task dataset and trained a classifier to infer an image's class label given only its hash. For this, we first computed the hashes of all samples in the dataset and trained a simple classifier that takes a 96-bit vector as input. The adversary only needs black-box access to the perceptual hashing function for this kind of attack.

Experimental Setup. We trained fully-connected neural networks with three hidden layers on hashes of the ImageNet train split with 1,000 classes and standard cross-entropy loss. To create a balanced dataset, we randomly picked 732 samples from each class resulting in a total of 732,000 training samples. We first computed the binary hashes for the samples using NeuralHash and used 90% of the hashes from each class for training and the remaining hashes for hyperparameter optimization. Since many of the ImageNet classes are quite similar, e.g., different dog breeds, we also used 85 coarse ImageNet categories [16] for training. We balanced the categories by randomly picking 980 samples from each category, resulting in 83,300 training samples, and again used 10% of it for hyperparameter optimization. In the following, we will refer to this task as categorization.

We evaluated the networks on the official ImageNet validation split containing 50,000 samples. For evaluating the categorization model, we randomly sampled 50 samples from each category from the validation split. As evaluation metrics, we computed the top-1, top-5, and top-10 prediction accuracy's mean and standard deviation on ten different training runs and dataset splits.

Results. We state our evaluation results in Table 3a. The results confirm our assumption that hash values indeed leak information about the corresponding images' contents. Our classifier trained on 1,000 classes achieved a top-1 test accuracy of 4.34%. Note that ImageNet is a quite challenging dataset and random guessing only achieves an expected accuracy of 0.1%. For the top-5 accuracy, the neural network reached an accuracy as high as 12.03%, and even 17.75% for the top-10 accuracy. The categorization model trained on the more coarse 85 categories predicted the correct category in 8.76% of the cases. Note that in this case prediction through random guessing would be 1.18%. For the top-5 and top-10 accuracy, the model even achieved 25.85% and 38.59% correct predictions. The standard deviation is quite low, so random selection of training and evaluation samples has only minor effects on the performance of the models. For further analysis, we calculated the share of correct predictions for each class and category for a randomly selected model and present the three classes and categories that have been predicted with the highest precision in Table 3b.



(a) Image Rotation (b) Brightness Changes (c) Contrast Changes

(d) Center Cropping (e) Downsizing (f) JPEG Compression

Figure 6: Robustness results (mean + std) for image transformations computed on all ImageNet training samples (Adversary 3). The mean Hamming distance could also be interpreted as the share of hash bits expected to change.

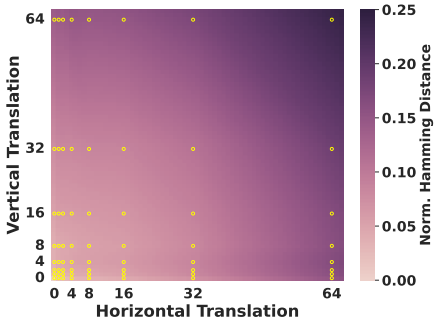


Figure 7: Mean Hamming distance for all ImageNet training images translated horizontally and vertically (Adversary 3). Yellow dots represent the distances for varying translations in both directions.

	Classification	Categorization
Top-1	04.34% ± 0.046%	08.76% ± 0.237%
Top-5	12.03% ± 0.090%	25.85% ± 0.423%
Top-10	17.75% ± 0.182%	38.59% ± 0.728%

(a) Test accuracy (mean + std).

Classification		Categorization	
Class	*Pre.*	*Category*	*Pre.*
Website	52.0%	Mosquito Net	52.0%
Panda	36.0%	Sloth	30.0%
Leonberg (dog breed)	36.0%	Hay	30.0%

(b) Share of correctly classified hashes.

Table 3: Evaluation results for our hash information extraction attack. Table 3a shows the mean accuracy measured on ten runs for the classification and categorization task. Random guessing has an expected top-1 accuracy of 0.1% and 1.2%, respectively. Table 3b shows the classes and categories, respectively, on which the hashes were classified correctly in the majority of the cases.

In conclusion, our simple ImageNet classifiers have shown that hashes still contain information about the images' contents. And even though the hashes contain less information than the corresponding images, they should be handled with care in practical applications. An attacker with access to a user's image hashes might infer private information about contents on the device.

7 LESSONS & IMPLICATIONS

Current systems are not robust. Our experimental results illustrate that perceptual hashing systems like NeuralHash are in no way robust against gradient-free detection evasion attacks. Simple image modifications, such as flipping an image horizontally, already allow an attacker to evade detection in most cases, even without direct access to the system. The attacker does not need any technical knowledge and can manipulate the images with standard image editors to evade detection. Even though we focused in our experiments on NeuralHash, we are convinced that other systems are also vulnerable to this kind of attack since deep neural networks react sensitively to input changes, as also adversarial attacks illustrate. As we outline later, the networks might be trained to be more robust against simple image manipulations but will never be entirely secure. A more robust system will force the attacker to alter the images stronger, but the content of the images will most

likely still be visible and recognizable. For example, disassembling an image into smaller parts for storing and reassembling the image only for viewing would lead to completely different hashes for each part without any loss of information in the full image.

On the other hand, evasion attacks exploiting gradient information allow evasion with only minor image manipulations. Our experiments have shown that with a bit of technical knowledge, an attacker can modify the images to evade detection without the modifications being clearly visible. Such attacks will most likely be possible in any neural network-based CSS system if the attacker has access to the model on the user device and is, therefore, able to calculate the gradients of the model.

Client-side scanning systems can be misused for malicious purposes. Proponents of client-side scanning emphasize the fact that the systems enable the detection of CSAM and other criminal material while avoiding backdoor keys in the end-to-end encryption and maintaining the user's privacy. However, client-side scanning opens the door for other malicious attacks. While evasion of the detection system only renders the system useless, an attacker can further misuse the system for framing or monitoring innocent users using hash collision attacks. As we have demonstrated, an attacker can slightly alter images to change their hashes to a specific value, causing false-positive detections for arbitrary image contents. With access to an official hash database or a surrogate, an attacker can frame innocent people by sending these manipulated images to their devices. As a result, without even knowing, the receivers of such images get flagged by the system.

As legislators call for preventing the distribution of harmful material while maintaining encryption gets louder, client-side scanning systems using perceptual hashing look very promising at first glance. However, governments or organizations with control over the client-side scanning system could share manipulated images in social media, and users that downloaded a subset of these images get flagged. Mass surveillance of people with undesired beliefs and opinions is therefore simply and secretly realizable. Additional tools or backdoors in the user devices are not needed. Also, there is no guarantee that the hash database will not be extended with additional, non-criminal content for surveillance. Since the databases are not publicly available, changes are not traceable, and the targeted content might be anything – who controls the provider of such a system?

Hash values still contain information. Even though the hash values of images contain much less information than the images themselves, our experiments have shown that there is still some information encoded in these hash values. Using the hash values, inference about the general content of images on the device is possible. Although image information is highly compressed, sharing these hashes with the system's provider might harm a user's privacy. Moreover, hashes with a larger number of bits are expected to leak even more information due to an increased information capacity. Apparently, Microsoft's PhotoDNA, which is already widely used for CSAM detection, produces hashes with 144-byte values [33]. We expect these hashes to leak even more information than NeuralHash, whose hashes consist of 96 bits.

Mitigating the risks. Regarding the technical aspects of the system, one way to mitigate the effectiveness of collision attacks against hashing-based detection systems is to install another server-side hashing procedure not available to the attacker. But this would, in turn, imply that the images are not encrypted on the server or could be decrypted by the provider, questioning the promised privacy advantages. Another method would be to restrict the model access and prevent gradient computations, which would indeed make gradient-based attacks hardly feasible. However, these steps do not improve the robustness against standard image transformations. Client-side scanning methods based on neural networks will most likely enable gradient computation or approximation and, consequently, facilitate arbitrary hash manipulations. Hence, even an updated version of NeuralHash's embedding network would very likely suffer from the demonstrated vulnerabilities.

To complicate hash manipulations, we expect training the embedding network in an adversarial setting [22, 46] might improve its robustness against attacks that manipulate the images. In adversarial training, a sample is first manipulated by one of our gradient-based attacks or standard image transformations and then used as a regular training sample to update the model's weights. It is further important to restrict public access to the hash database. If the plain hashes are leaked, it is even easier for an adversary to produce a large number of false-positive matches. On the other hand, an independent instance should monitor the database to avoid manipulations and the introduction of non-criminal material.

In summary, our work points out that NeuralHash, and arguably deep perceptual hashing algorithms in general, are not robust and susceptible against gradient-based and gradient-free image manipulations. It is questionable if neural networks in their current form will ever be fully robust. We further share the viewpoint that client-side scanning might have initially reasonable goals but also acknowledge the potential risks of expanding the scope of application since it is running on all devices regardless of whether crime is suspected or not. We hope that service providers and governments are not going in this direction in crime detection due to the many technical and societal risks.

From a technical and ethical viewpoint, we conclude that NeuralHash and related client-side scanning systems do not provide a safe method for detecting legal violations and should not be deployed on user devices since attackers, service providers, and governments could simply manipulate and misuse the systems according to their interests. After all, mobile devices contain a lot of sensitive information about their users, ranging from dating behavior to health care and financial status. Instead, based on our results, we call for NeuralHash and other client-side scanning systems not to be installed in practice, due to their manipulability, risk of abuse, and lack of robustness. Granting access to the content stored on the devices, even only the images, might pave the way for further interventions in a user's privacy. We should, therefore, be very careful in deploying detection systems directly on user devices.

8 CONCLUSION

Using NeuralHash as a practical example from a real-world use case, our work demonstrated that perceptual hashing-based detection systems might be cryptographically well-proven but are still highly susceptible to various attacks, some of them trivial, that crack the system. We empirically showed that the system is vulnerable to collision attacks that might lead to serious consequences for privacy and allow monitoring of users beyond the intended use case of CSAM detection. Our results further suggest that the feature extraction step in the hash computation is in no way robust against gradient- and transformation-based image manipulations and induces vulnerabilities that allow an attacker to easily evade detection. Moreover, even though NeuralHash produces only short bit-string hashes, we practically demonstrated that the hashes still leak some information on the images' contents and should be treated with caution. We conclude that NeuralHash, and deep perceptual hashing systems in general, pose several risks to the reliability of the systems and even to human society. Therefore, we believe that NeuralHash, and other client-side detection systems, should not be installed in their current state.

Disclaimer. The views and opinions expressed in this article are those of the authors and do not reflect the official policy or position of the authors' institutions or any agency of the German Federal Government.

Reproducibility Statement We deliberately decided to make our source code publicly available to reproduce the experiments and investigate existing and future perceptual hashing systems: https://github.com/ml-research/Learning-to-Break-Deep-Perceptual-Hashing.

ACKNOWLEDGMENTS

This work was supported by the German Ministry of Education and Research (BMBF) within the framework program "Research for Civil Security" of the German Federal Government, project KISTRA (reference no. 13N15343). It also benefited from the National Research Center for Applied Cybersecurity ATHENE, a joint effort of BMBF and the Hessian Ministry of Higher Education, Research, Science and the Arts (HMWK). We thank the anonymous reviewers for their insightful and constructive comments on this paper. Also, we thank Elissa Redmiles for her valuable feedback.

REFERENCES

[1] Hal Abelson, Ross Anderson, Steven M. Bellovin, Josh Benaloh, Matt Blaze, Jon Callas, Whitfield Diffie, Susan Landau, Peter G. Neumann, Ronald L. Rivest, Jeffrey I. Schiller, Bruce Schneier, Vanessa Teague, and Carmela Troncoso. 2021. Bugs in our Pockets: The Risks of Client-Side Scanning. *CoRR* abs/2110.07450 (2021).

[2] Mathias Appel. 2017. PIXNIO-226632-4681x3100.jpeg. https://pixnio.com/creative-commons-license, accessed Sept. 23, 2021; Cropped; License: https://pixnio.com/creative-commons-license.

[3] Anish Athalye. 2021. NeuralHash Collider. https://github.com/anishathalye/neural-hash-collider, accessed Oct. 12, 2021.

[4] Jiawang Bai, Bin Chen, Yiming Li, Dongxian Wu, Weiwei Guo, Shu-tao Xia, and En-hui Yang. 2020. Targeted Attack for Deep Hashing based Retrieval. In *ECCV*.

[5] Mihir Bellare. 2021. The Apple PSI Protocol. https://www.apple.com/child-safety/pdf/Technical_Assessment_of_CSAM_Detection_Mihir_Bellare.pdf, accessed: Oct. 1, 2021.

[6] Patrick Breyer, Alviina Alametsä, Rosa D'Amato, Pernando Barrena, Saskia Bricmont, Antoni Comín, Gwendoline Delbos-Corfield, Francesca Donato, Cornelia Ernst, Claudia Gamon, Markéta Gregorová, Francisco Guerreiro, Svenja Hahn, Irena Joveva, Petra Kammerevert, Marcel Kolaja, Moritz Körner, Karen Melchior, Clara Ponsatí, and Mikuláš Peksa. 2021. Cross-Party Letter of Member of the European Parliament Against General Monitoring. https://www.patrick-breyer.de/wp-content/uploads/2021/11/20211020_Letter_General_Monitoring.pdf, accessed: Nov. 19, 2021.

[7] John F. Canny. 1986. A Computational Approach to Edge Detection. *IEEE TPAMI* 8, 6 (1986), 679–698.

[8] Nicholas Carlini and David A. Wagner. 2017. Towards Evaluating the Robustness of Neural Networks. In *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*. IEEE Computer Society, 39–57. https://doi.org/10.1109/SP.2017.49

[9] European Commission. 2020. EU strategy for a more effective fight against child sexual abuse. https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:52020DC0607, accessed: Nov. 16, 2021.

[10] Francesco Croce and Matthias Hein. 2020. Minimally distorted Adversarial Examples with a Fast Adaptive Boundary Attack. In *Proceedings of the 37th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 119)*. 2196–2205.

[11] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *CVPR*. 248–255.

[12] Brian Dolhansky and Cristian Canton-Ferrer. 2020. Adversarial collision attacks on image hashing functions. *CoRR* abs/2011.09473 (2020).

[13] Xiaoyi Dong, Dongdong Chen, Jianmin Bao, Chuan Qin, Lu Yuan, Weiming Zhang, Nenghai Yu, and Dong Chen. 2020. GreedyFool: Distortion-Aware Sparse Adversarial Attack. In *Conference on Neural Information Processing Systems (NeurIPS)*, Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.).

[14] Yinpeng Dong, Fangzhou Liao, Tianyu Pang, Hang Su, Jun Zhu, Xiaolin Hu, and Jianguo Li. 2018. Boosting Adversarial Attacks With Momentum. In *2018 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 9185–9193.

[15] Andrea Drmic, Marin Silic, Goran Delac, Klemo Vladimir, and Adrian S. Kurdija. 2017. Evaluating robustness of perceptual image hashing algorithms. In *2017 40th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*. 995–1000. https://doi.org/10.23919/MIPRO.2017.7973569

[16] Noam Eshed. 2020. *Novelty detection and analysis in convolutional neural networks*. Master's thesis. Cornell University.

[17] Facebook. 2019. Open-Sourcing Photo- and Video-Matching Technology to Make the Internet Safer. https://about.fb.com/news/2019/08/open-source-photo-video-matching, accessed Oct. 18, 2021.

[18] Department for Digital, Culture, Media & Sport, United Kingdom Government. 2021. Draft Online Safety Bill. https://assets.publishing.service.gov.uk/government/uploads/system/uploads/attachment_data/file/985033/Draft_Online_Safety_Bill_Bookmarked.pdf, accessed: Nov. 16, 2021.

[19] National Center for Missing & Exploited Children. 2021. CyberTipline - By the Numbers. https://www.missingkids.org/gethelpnow/cybertipline, accessed: Oct. 1, 2021.

[20] David Forsyth. 2021. Apple's CSAM detection technology. https://www.apple.com/child-safety/pdf/Technical_Assessment_of_CSAM_Detection_David_Forsyth.pdf, accessed: Oct. 1, 2021.

[21] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *VLDB*. 518–529.

[22] Ian Goodfellow, Jonathon Shlens, and Christian Szegedy. 2015. Explaining and Harnessing Adversarial Examples. In *ICLR*.

[23] R. Hadsell, S. Chopra, and Y. LeCun. 2006. Dimensionality Reduction by Learning an Invariant Mapping. In *CVPR*, Vol. 2. 1735–1742.

[24] Qingying Hao, Licheng Luo, Steve TK Jan, and Gang Wang. 2021. It's Not What It Looks Like: Manipulating Perceptual Hashing based Applications. In *Proceedings of The ACM Conference on Computer and Communications Security (CCS)*.

[25] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, Quoc V. Le, and Hartwig Adam. 2019. Searching for MobileNetV3. In *ICCV*.

[26] Apple Inc. 2021. CSAM Detection - Technical Summary. https://www.apple.com/child-safety/pdf/CSAM_Detection_Technical_Summary.pdf, accessed: Sept. 22, 2021.

[27] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *ACM - STOC*. 604–613.

[28] Shubham Jain, Ana-Maria Cretu, and Yves-Alexandre de Montjoye. 2021. Adversarial Detection Avoidance Attacks: Evaluating the robustness of perceptual hashing-based client-side scanning. *CoRR* abs/2106.09820 (2021).

[29] Aditya Khosla, Nityananda Jayadevaprakash, Bangpeng Yao, and Li Fei-Fei. 2011. Novel Dataset for Fine-Grained Image Categorization. In *First Workshop on Fine-Grained Visual Categorization, CVPR*.

[30] Lim Swee Kiat. 2021. apple-neuralhash-attack. https://github.com/greentfrapp/apple-neuralhash-attack, accessed Oct. 12, 2021.

[31] Yannic Kilcher. 2021. Neural Hash Collision Creator. https://github.com/yk/neural_hash_collision, accessed Oct. 12, 2021.

[32] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *ICLR*.

[33] Neal Krawetz. 2021. PhotoDNA and Limitations. https://hackerfactor.com/blog/index.php?/archives/931-PhotoDNA-and-Limitations.html, accessed: Jan. 13, 2022.
[34] Sarah Jamie Lewis. 2021. Tweets on NeuralHash. https://twitter.com/SarahJamieLewis/status/1428146453394821125, accessed: Oct. 1, 2021.
[35] Venice Erin Liong, Jiwen Lu, Gang Wang, Pierre Moulin, and Jie Zhou. 2015. Deep hashing for compact binary codes learning. In *CVPR*. 2475–2483.
[36] Haomiao Liu, Ruiping Wang, Shiguang Shan, and Xilin Chen. 2016. Deep Supervised Hashing for Fast Image Retrieval. In *CVPR*. 2064–2072.
[37] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. 2018. Towards Deep Learning Models Resistant to Adversarial Attacks. In *ICLR*.
[38] Microsoft. 2015. PhotoDNA. https://www.microsoft.com/en-us/photodna, accessed Oct. 18, 2021.
[39] Official Journal of the European Union. 2021. Regulation (EU) 2021/1232. https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32021R1232, accessed: Nov. 16, 2021.
[40] Nicolas Papernot, Patrick D. McDaniel, Somesh Jha, Matt Fredrikson, Z. Berkay Celik, and Ananthram Swami. 2016. The Limitations of Deep Learning in Adversarial Settings. In *IEEE European Symposium on Security and Privacy (EuroS&P)*. 372–387.
[41] Benny Pinkas. 2021. A Review of the Cryptography Behind the Apple PSI System. https://www.apple.com/child-safety/pdf/Technical_Assessment_of_CSAM_Detection_Benny_Pinkas.pdf, accessed: Oct. 1, 2021.
[42] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. 2015. ImageNet Large Scale Visual Recognition Challenge. *IJCV* 115, 3 (2015), 211–252.
[43] Edward Snowden. 2021. The All-Seeing "i": Apple Just Declared War on Your Privacy. https://edwardsnowden.substack.com/p/all-seeing-i, accessed: Oct. 1, 2021.
[44] Jiawei Su, Danilo Vasconcellos Vargas, and Kouichi Sakurai. 2019. One Pixel Attack for Fooling Deep Neural Networks. *IEEE TEVC* 23, 5 (2019), 828–841.
[45] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. 2014. Intriguing properties of neural networks. In *ICLR*.
[46] Xunguang Wang, Zheng Zhang, Guangming Lu, and Yong Xu. 2021. Targeted Attack and Defense for Deep Hashing. In *ACM SIGIR*. 2298–2302.
[47] Xunguang Wang, Zheng Zhang, Baoyuan Wu, Fumin Shen, and Guangming Lu. 2021. Prototype-Supervised Adversarial Network for Targeted Attack of Deep Hashing. In *CVPR*. 16357–16366.
[48] Yongwei Wang, Hamid Palangi, Z. Jane Wang, and Haoqian Wang. 2018. RevHashNet: Perceptually de-hashing real-valued image hashes for similarity retrieval. *Signal Processing: Image Communication* 68 (2018), 68–75. https://doi.org/10.1016/j.image.2018.06.018
[49] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing* 13, 4 (2004), 600–612.
[50] Katie Crampton (WMUK). 2020. Black Lives Matter protest at US Embassy. https://commons.wikimedia.org/wiki/File:Black_Lives_Matter_protest_at_US_Embassy,_London_01.jpg, accessed Sept. 23, 2021; Cropped; License: https://creativecommons.org/licenses/by-sa/4.0/deed.en.
[51] Dayan Wu, Zheng Lin, Bo Li, Mingzhen Ye, and Weiping Wang. 2017. Deep Supervised Hashing for Multi-Label and Large-Scale Image Retrieval. In *ACM ICMR*. 150–158.
[52] Yanru Xiao and Cong Wang. 2021. You See What I Want You To See: Exploring Targeted Black-Box Transferability Attack for Hash-Based Image Retrieval Systems. In *CVPR*. 1934–1943.
[53] Erkun Yang, Tongliang Liu, Cheng Deng, and Dacheng Tao. 2020. Adversarial Examples for Hamming Space Search. *IEEE Transactions on Cybernetics* 50, 4 (2020), 1473–1484.
[54] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. 2018. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 586–595.
[55] F. Zhao, Yongzhen Huang, Liang Wang, and Tieniu Tan. 2015. Deep semantic ranking based hashing for multi-label image retrieval. In *CVPR*. 1556–1564.
[56] Mingkang Zhu, Tianlong Chen, and Zhangyang Wang. 2021. Sparse and Imperceptible Adversarial Attack via a Homotopy Algorithm. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, Marina Meila and Tong Zhang (Eds.), Vol. 139. 12868–12877.